

Documentacion de un Sistema de Gestion para una Tienda Online

Beatriz Garcia Lopez

13 de julio de 2026

1. Introducción

Se nos ha solicitado diseñar y construir la base de datos completa para gestionar las operaciones de una tienda online, asegurando que la información de productos, clientes y ventas esté organizada y protegida.

2. Descripción del problema

Lo que hace este sistema es resolver el desorden en el inventario y las ventas. Permite registrar qué compró cada cliente, saber exactamente cuántos productos quedan disponibles y evitar que se vendan artículos agotados.

3. Análisis y Diseño

En este apartado vamos a estructurar un poco las tablas y implementar un DER(Diagrama Entidad-Relacion) utilizando la página "Draw.io"

TABLAS	¿QUE ORGANIZA?
Categorias	Organiza los productos en 5 categorias diferentes (telefonos, laptops, accesorios, tablets y relojes inteligentes)
Reseñas	Registra las reseñas de los productos y el ranking del mismo, el cual se evalua de 1 a 5 estrellas, siendo 1 = muy malo y 5 = excelente. Un detalle importante es que las reseñas solo podran ser realizadas por clientes que hayan comprado un producto, sin importar la categoria del mismo.
Pedidos	Especifica los detalles especificos del producto; como fecha de pedido y el estado del mismo (Enviado, Pendiente y Entregado).
Detalles del Pedido	Guarda los datos especificos del pedido como por ejemplo la cantidad (¿cuantos productos se van a enviar?) y el precio unitario del producto.
Productos	Esta tabla es la mas importante ya que desde aqui vamos a poder gestionar todas las demas tablas. Es como un "gestor que se conecta con todo lo demas". De igual manera, aqui tenemos el nombre, descripcion, precio y stock de todos los productos disponibles al momento.
Cientes	En esta tabla esta toda la informacion personal del cliente como por ejemplo nombre, correo, telefono y direccion.

A continuacion tenemos un diagrama ER explicando de mejor manera la tabla

3.1. Diagrama Entidad-Relacion

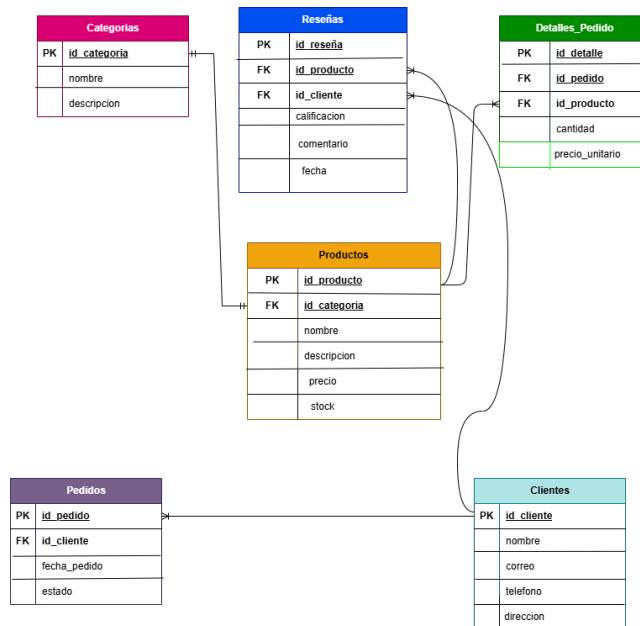


Figura 1: DER de la Base de Datos

3.2. Esquema en tercera forma normal (3NF), con justificación de normalización.

Primera Forma Normal (1NF): Todas las tablas tienen datos atómicos. Esto significa que no tenemos listas con datos mezclados en una sola celda; cada campo guarda un solo dato. Por ejemplo, en la tabla **Clientes**, el nombre, correo y teléfono están separados en sus propios campos, en lugar de tener un solo campo saturado de "datos de contacto".

Segunda Forma Normal (2NF): Todos los datos de una tabla dependen por completo de su llave principal(PK). Un excelente ejemplo del diseño es la tabla **Detalles_Pedido**: la cantidad de productos comprados depende exactamente de esa línea de compra en específico (id_detalle), no solo del pedido ni del producto por sí solos.

Tercera Forma Normal (3NF): Ningún dato depende de otro que no sea la llave principal. Tenemos dos ejemplos:

- En la tabla **Pedidos**, no pusimos el nombre del cliente dentro de la tabla, porque el nombre depende del cliente (por eso tiene su propia tabla), no del pedido en sí. Solo los unimos con la llave foránea id_cliente".

En la tabla **Productos**, no fue necesario escribir el nombre o la descripción de la categoría. Solo usamos id_categoria. Así, si la categoría cambia de nombre, se actualiza en su propia tabla y no crea inconsistencias en los productos.

3.3. Identificación de Claves

A continuación se detallan las claves primarias, foráneas y candidatas de cada entidad dentro del modelo relacional del proyecto:

Claves Primarias (PK)

Las claves primarias seleccionadas actúan como identificadores únicos e irrepetibles para cada registro dentro de su respectiva tabla:

- **Categorías:** id_categoria
- **Productos:** id_producto
- **Clientes:** id_cliente
- **Pedidos:** id_pedido
- **Detalles_Pedido:** id_detalle
- **Reseñas:** id_reseña

Llaves Foráneas (FK)

Las llaves foráneas establecen la integridad referencial y las relaciones entre las entidades del sistema:

- **Productos:** id_categoria (referencia a la tabla Categorías)
- **Pedidos:** id_cliente (referencia a la tabla Clientes)
- **Detalles_Pedido:** id_pedido (referencia a la tabla Pedidos) y id_producto (referencia a la tabla Productos)
- **Reseñas:** id_producto (referencia a la tabla Productos) y id_cliente (referencia a la tabla Clientes)

Llaves Candidatas

Las llaves candidatas son aquellos atributos que también poseen la cualidad de unicidad y podrían funcionar como identificadores principales, aunque se haya optado por un ID numérico por eficiencia:

- **Clientes:** El atributo correo es una llave candidata fuerte, ya que en el sistema de ventas no deberían existir dos clientes registrados con la misma dirección de correo electrónico.
- **Categorías:** El atributo nombre funciona como llave candidata, ya que no tendría sentido comercial tener dos categorías llamadas exactamente igual.

4. Implementacion de la Base de Datos

En esta sección se presenta el script correspondiente a la creación del esquema relacional, la definición de llaves primarias y foráneas, así como las restricciones de integridad y la creación de índices para optimizar las consultas. Finalmente, se incluye el código utilizado para poblar la base de datos con registros de prueba

```
1 -- 1. Crear esquema y limpiar versiones viejas
2 CREATE SCHEMA IF NOT EXISTS proyecto_final_bd;
3 USE proyecto_final_bd;
4 DROP TABLE IF EXISTS Resenas, Detalles_Pedido, Pedidos, Productos,
   Clientes, Categorías;
5
6 -- 2. Crear las tablas principales
7 CREATE TABLE Categorías (id_categoria INT PRIMARY KEY, nombre
   VARCHAR(50), descripcion VARCHAR(200));
8 CREATE TABLE Clientes (id_cliente INT PRIMARY KEY, nombre VARCHAR
   (100), correo VARCHAR(100) UNIQUE, telefono VARCHAR(20),
   direccion VARCHAR(200));
9 CREATE TABLE Productos (id_producto INT PRIMARY KEY, nombre VARCHAR
   (100), descripcion VARCHAR(200), precio DECIMAL(10,2), stock
   INT CHECK (stock >= 0), id_categoria INT, FOREIGN KEY (
   id_categoria) REFERENCES Categorías(id_categoria));
```

```

10 CREATE TABLE Pedidos (id_pedido INT PRIMARY KEY, id_cliente INT,
    fecha_pedido DATE, estado VARCHAR(20), FOREIGN KEY (id_cliente)
    REFERENCES Clientes(id_cliente));
11 CREATE TABLE Detalles_Pedido (id_detalle INT PRIMARY KEY, id_pedido
    INT, id_producto INT, cantidad INT, precio_unitario DECIMAL
    (10,2), FOREIGN KEY (id_pedido) REFERENCES Pedidos(id_pedido),
    FOREIGN KEY (id_producto) REFERENCES Productos(id_producto));
12 CREATE TABLE Resenas (id_resena INT PRIMARY KEY, id_producto INT,
    id_cliente INT, calificacion INT, comentario VARCHAR(255),
    fecha DATE, FOREIGN KEY (id_producto) REFERENCES Productos(
    id_producto), FOREIGN KEY (id_cliente) REFERENCES Clientes(
    id_cliente));
13
14 -- 3. Crear indices para agilizar busquedas
15 CREATE INDEX idx_producto_nombre ON Productos(nombre);
16 CREATE INDEX idx_producto_categoria ON Productos(id_categoria);
17 CREATE INDEX idx_pedido_cliente ON Pedidos(id_cliente);
18
19 -- 4. Insertar datos (Muestra representativa)
20 INSERT IGNORE INTO Categorias (id_categoria, nombre, descripcion)
    VALUES (1, 'Laptops', 'Computadoras portatiles');
21 INSERT IGNORE INTO Clientes (id_cliente, nombre, correo, telefono,
    direccion) VALUES (1, 'Juan Perez', 'juan@email.com', '
    5551112222', 'Calle 1');

```

Listing 1: Creación de Base de Datos

The screenshot shows the MySQL Workbench interface. The 'Pedidos' table structure is visible in the 'Structure' tab, showing columns: id_pedido (INT, PRIMARY KEY), id_cliente (INT), fecha_pedido (DATE), estado (VARCHAR(20)), and foreign key constraints. Below the table structure, the 'Queries' tab shows a list of SQL queries executed, including the creation of tables and the insertion of data.

Query	Result
1. CREATE TABLE Pedidos (id_pedido INT PRIMARY KEY, id_cliente INT, fecha_pedido DATE, estado VARCHAR(20), FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente));	Query OK, 0 rows affected
2. CREATE TABLE Detalles_Pedido (id_detalle INT PRIMARY KEY, id_pedido INT, id_producto INT, cantidad INT, precio_unitario DECIMAL(10,2), FOREIGN KEY (id_pedido) REFERENCES Pedidos(id_pedido), FOREIGN KEY (id_producto) REFERENCES Productos(id_producto));	Query OK, 0 rows affected
3. CREATE TABLE Resenas (id_resena INT PRIMARY KEY, id_producto INT, id_cliente INT, calificacion INT, comentario VARCHAR(255), fecha DATE, FOREIGN KEY (id_producto) REFERENCES Productos(id_producto), FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente));	Query OK, 0 rows affected
4. CREATE INDEX idx_producto_nombre ON Productos(nombre);	Query OK, 0 rows affected
5. CREATE INDEX idx_producto_categoria ON Productos(id_categoria);	Query OK, 0 rows affected
6. CREATE INDEX idx_pedido_cliente ON Pedidos(id_cliente);	Query OK, 0 rows affected
7. INSERT IGNORE INTO Categorias (id_categoria, nombre, descripcion) VALUES (1, 'Laptops', 'Computadoras portatiles');	Query OK, 1 row affected
8. INSERT IGNORE INTO Clientes (id_cliente, nombre, correo, telefono, direccion) VALUES (1, 'Juan Perez', 'juan@email.com', '5551112222', 'Calle 1');	Query OK, 1 row affected

5. Consultas y Procedimientos Almacenados

```
1  -- Consultar clientes con pedidos pendientes
2  SELECT c.nombre, COUNT(p.id_pedido) AS total_pendientes
3  FROM Clientes c JOIN Pedidos p ON c.id_cliente = p.id_cliente
4  WHERE p.estado = 'Pendiente' GROUP BY c.id_cliente;
5
6  -- Procedimiento: Registrar pedido limitando a 5 pendientes
7  DROP PROCEDURE IF EXISTS sp_registrar_pedido;
8  DELIMITER //
9  CREATE PROCEDURE sp_registrar_pedido(IN p_cliente INT)
10 BEGIN
11     DECLARE v_pendientes INT;
12     DECLARE v_id INT;
13     SELECT COUNT(*) INTO v_pendientes FROM Pedidos WHERE id_cliente
14         = p_cliente AND estado = 'Pendiente';
15
16     IF v_pendientes < 5 THEN
17         SELECT IFNULL(MAX(id_pedido), 0) + 1 INTO v_id FROM Pedidos
18         ;
19         INSERT INTO Pedidos (id_pedido, id_cliente, fecha_pedido,
20             estado) VALUES (v_id, p_cliente, CURDATE(), 'Pendiente');
21     ELSE
22         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Limite
23             alcanzado';
24     END IF;
25 END //
26 DELIMITER ;
```

Listing 2: Lógica de Negocio

```

1  -- Consulta 1: Productos por Categoría
2  • SELECT c.nombre AS categoria, p.nombre AS producto, p.precio, p.stock
3  FROM Productos p
4  JOIN Categorías c ON p.id_categoria = c.id_categoria
5  WHERE p.stock > 0
6  ORDER BY c.nombre ASC, p.precio ASC;

```

categoria	producto	precio	stock
Accesorios	Mica Cristal	100.00	120
Accesorios	Cable USB C	150.00	100
Accesorios	Funda iPhone	200.00	60
Accesorios	Mouse inalámbrico	250.00	50
Accesorios	Cargador Universal	300.00	50
Accesorios	Teclado USB	300.00	40
Accesorios	Base Laptop	400.00	20
Accesorios	Powerbank	500.00	45
Accesorios	Webcam	600.00	30
Accesorios	Microfono	800.00	15
Audio	Bocina JBL	1200.00	35
Audio	Audifonos Sony	1500.00	40
Audio	Bocina LG	2000.00	15
Audio	AirPods	3000.00	20
Audio	Audifonos Bose	4500.00	10
Laptops	Laptop Lenovo	11000...	12
Laptops	Laptop HP	12000...	10
Laptops	Laptop Asus	14500...	6
Laptops	Laptop Dell	15000...	5
Laptops	MacBook Air	18000...	8
Monitores	Monitor Acer	2500.00	25
Monitores	Monitor Samsung	3000.00	20
Monitores	Monitor Dell	3200.00	10
Monitores	Monitor LG	4000.00	15
Monitores	Monitor Asus	4500.00	8
Smartphones	Motorola G	5000.00	30
Smartphones	Huawei Edge	5500.00	10
Smartphones	Xiaomi Redmi	6000.00	25
Smartphones	Samsung S22	14000...	20
Smartphones	iPhone 13	16000...	15

Figura 2: Resultado de la consulta 1: Productos por categoría.

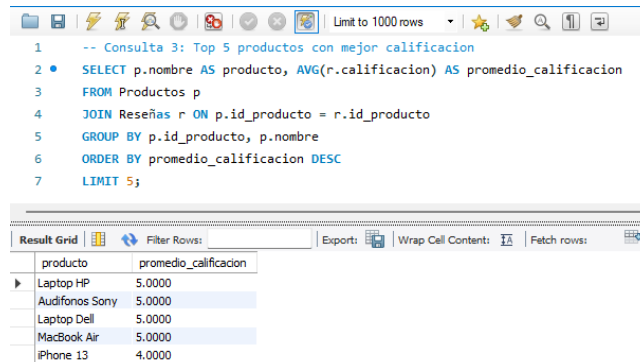
```

1  -- Consulta 2: Clientes con pedidos pendientes
2  • SELECT c.nombre AS cliente, COUNT(p.id_pedido) AS total_pedidos_pendientes
3  FROM Clientes c
4  JOIN Pedidos p ON c.id_cliente = p.id_cliente
5  WHERE p.estado = 'Pendiente'
6  GROUP BY c.id_cliente, c.nombre;

```

cliente	total_pedidos_pendientes
Ana Martinez	1
Diego Morales	1
Ricardo Vargas	1
Carlos Lopez	1

Figura 3: Resultado de la consulta 2: Clientes con pedidos pendientes.



```

1  -- Consulta 3: Top 5 productos con mejor calificación
2  • SELECT p.nombre AS producto, AVG(r.calificacion) AS promedio_calificacion
3    FROM Productos p
4   JOIN Reseñas r ON p.id_producto = r.id_producto
5   GROUP BY p.id_producto, p.nombre
6   ORDER BY promedio_calificacion DESC
7   LIMIT 5;

```

producto	promedio_calificacion
Laptop HP	5.0000
Audifonos Sony	5.0000
Laptop Dell	5.0000
MacBook Air	5.0000
iPhone 13	4.0000

Figura 4: Resultado de la consulta 3: Top 5 productos mejor calificados.

6. Validación y Optimización

Para comprobar que la lógica funciona, ejecutamos el procedimiento de reporte de stock mediante el comando CALL, obteniendo los datos esperados.

```

1  • USE proyecto_final_bd;
2  -- Procedimiento 1: Regitara un nuevo pedido exitoso, el cual inserta
3  -- el pedido 21 para el cliente 1
4  • CALL sp_registrar_pedido(21, 1, 1, 1, 26);

```

162	21:24:58	CALL sp_registrar_pedido(21, 1, 1, 1, 26)	1 row(s) affected
-----	----------	---	-------------------

Figura 5: Ejecución exitosa del procedimiento almacenado (Reporte de stock).

Para validar la optimización, usamos la instrucción EXPLAIN. La base de datos utilizó correctamente nuestro índice `idx_producto_nombre`, encontrando la Laptop en una sola lectura (**rows = 1**).

```

9  -- Prueba de Optimización: Uso de índices (Comprobación)
10 • EXPLAIN SELECT * FROM Productos WHERE nombre = 'Laptop HP';

```

id	select_type	table	partitions	type	possible_keys	key	key_len
1	SIMPLE	Productos	NULL	ref	idx_producto_nombre	idx_producto_nombre	403

#	Time	Action
1	21:52:26	USE proyecto_final_bd
2	21:52:26	CALL sp_reporte_stock_bajo()
3	21:52:26	CALL sp_registrar_pedido(1)
4	21:52:26	EXPLAIN SELECT * FROM Productos WHERE nombre = 'Laptop HP'

Figura 6: Uso de índices en búsqueda rápida.

7. Errores y Correcciones

Problema	Causa	Solución
Error 1054: Unknown column	Nombres de llaves foráneas automáticos y confusos.	Estandarizar la creación de tablas con nombres limpios (ej. <code>id_cliente</code>).
Warning 1366	Insertar datos sin especificar el orden de las columnas.	Declarar las columnas explícitamente en cada inserción.
Error 1452: Foreign key fails	Guardar detalles de un pedido antes que el producto.	Respetar la jerarquía insertando primero los datos padre (categorías, productos).
Error 1318: Incorrect arguments	Actualizar un procedimiento sin borrar el viejo de la memoria.	Usar <code>DROP PROCEDURE IF EXISTS</code> al inicio.

Cuadro 1: Resolución de problemas técnicos.

8. Conclusión

Desarrollar este sistema me permitió entender que una base de datos no es solo guardar información, sino conectarla con lógica real. Aprendí a estructurar las tablas usando normalización, a proteger la información con restricciones y a automatizar procesos con procedimientos almacenados, logrando un proyecto totalmente funcional y libre de errores.

9. Material de Apoyo Consultado

- Monroy, G. (2026). *Procedimientos Almacenados, Vistas y Triggers en MySQL*. Material de clase.
- Parzibyte - programador. (s. f.). <https://parzibyte.me/>
- Ivan. (2011, 25 diciembre). Crear BD en MySQL Workbench: Guía paso a paso para principiantes MySQL YA. Mysql YA. <https://mysqlya.com.ar/tutoriales/crear-bd-en-mysql-workbench/>
- Andrés, J. (s. f.). DISEÑO y CREACIÓN DE BASE DE DATOS CON MYSQL WORKBENCH (BD INVENTARIO). <https://base-de-datos-online.blogspot.com/2016/01/disen-y-creacion-de-base-de-datos-con.html>

- Parzibyte. (2021, 20 enero). MySQL - Inner join con count usando subconsulta. Parzibyte. <https://parzibyte.me/blog/posts/mysql-inner-join-count-subconsulta/>